

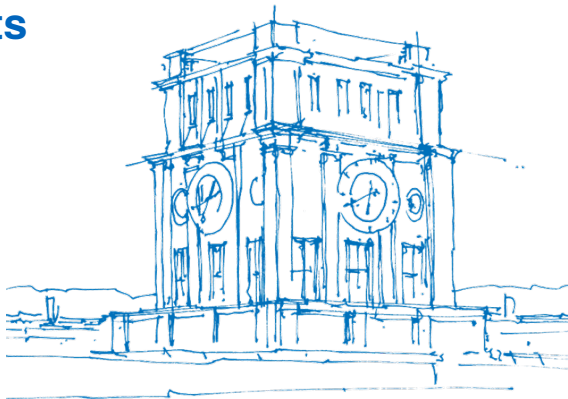
Autonomous Coding Agents

A seminar survey of agentic AI
for code generation — SS26

Thua Duc Nguyen

Supervisor: Derui Zhu
TUM School of Computation, Information and
Technology, Technical University of Munich

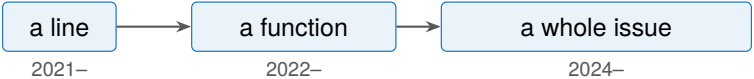
July 27, 2026



How we write code has changed

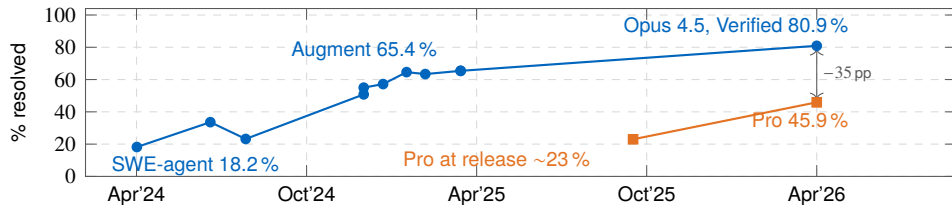


	Machine	Human	Checked by
Manual	Compiles and runs it	Writes every line	The test suite
Autocomplete 2021–	Suggests the next lines	Accepts or rejects each	You, line by line
One-shot 2022–	Writes a whole function	Specifies, reads all of it	Nothing
Agentic 2024–	Finds files, edits, runs tests	States intent, reviews outcome	Tests, <i>during</i> generation



Real progress

SWE-bench Verified progress — 500 human-screened GitHub issues, graded by the project's own tests.



Blue: **SWE-bench Verified**. Orange: **SWE-bench Pro** — harder, released Sept 2025.

Massive improvement in two years, and the models were not trained to be agents.

More on SWE-bench later.

What this talk shows

1. How do the agentic AI systems work?

Five stages, orchestration and interface, planning and refinement, five systems.

2. How do we measure performance?

The benchmark, the metrics, what the numbers hide.

3. Why are we stuck on the hard tasks?

Four reasons — and the pattern behind them.

A few terms, up front

Scaffold	The non-model code around it — tools, prompts, control flow
Tool call	A single scaffold action the model invokes and waits on — open a file, run a command, apply an edit
Patch	A description of a change — which lines, in which files — not the changed code
Harness	The fixed environment a submission is graded in — not the one it was made in
Oracle	Something outside the model that can check it — a test suite, a compiler, a human
Critic	A separate model, trained to score candidate outputs

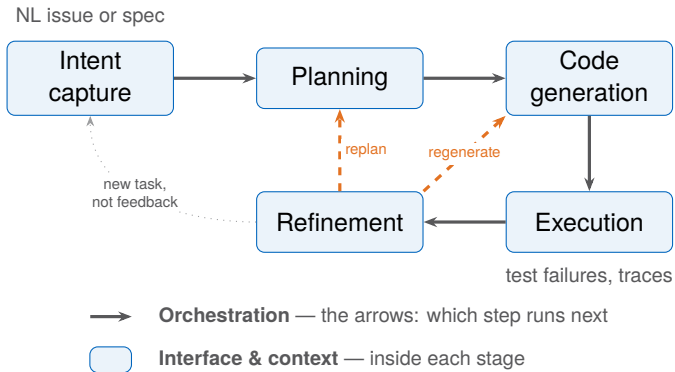
- 1 How do the agentic AI systems work?
- 2 How do we measure performance?
- 3 Why are we stuck on the hard tasks?

What is an agentic coding system?

A system that, given a natural-language request, **autonomously** produces a working code change — navigating files, editing, executing, and revising without step-by-step human guidance.

- **Autonomous** — decides what to do next, not told
- **Grounded in execution** — runs tests, reads errors, acts on them
- **Repository-scale** — works across files in a real codebase, not on isolated functions

The agent loop



How the loop is controlled

The five stages are **what** happens. Two controls decide **how** the loop runs — and each acts on a different part of it:

	Orchestration	Interface & context
Acts on	the arrows between stages	inside each stage
What it does	decides the next step	sets what it can see and do

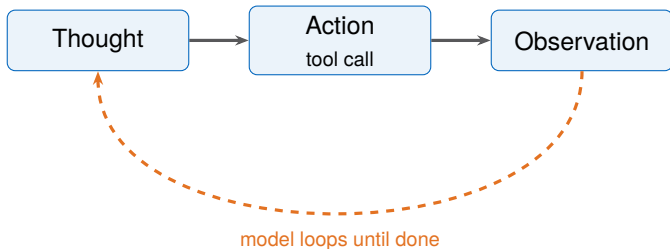
Neither is a stage — both cut *across* all five, and decide whether the loop even closes.

Two common ways to run the same five stages.

Mode	How it works	Who decides
Agentic loop ReAct [57]	Thought, Action, Observation; repeat	The model
Fixed pipeline [50]	Localise, repair, validate; no loop	The scaffold

Orchestration: agentic loop

The **model** is in control: the **ReAct** loop — Thought, Action, Observation — repeated until *the model* decides it is done [57].



Read a file, run tests, edit — **any tool, any order**, as many rounds as it takes. Nothing outside the model fixes the sequence.

Orchestration: fixed pipeline

The **scaffold** is in control: the steps are **fixed in advance**; the model only fills each one [50].



On failure it stops or moves on — **no arrow goes back**. Cheaper and predictable, but no recovery.

Interface and Context

What the agent sees and does.

Agent-Computer Interface (ACI)

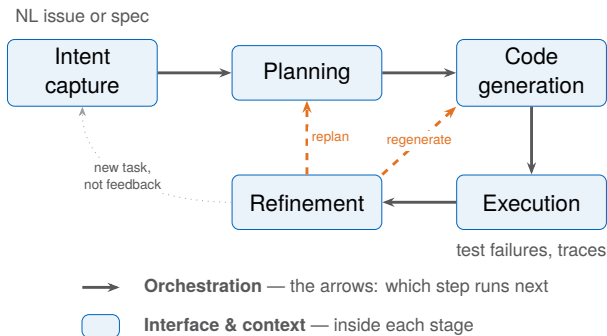
- Purpose-built command set
- Scrollable file viewer, linter-guarded edits, summarised search
- The scaffold controls what the model can see
- Malformed actions rejected before they run

Code as action

- The agent writes Python or bash in a sandbox
- Full shell access, no guardrails, no summarisation
- The model manages its own context

	ACI [54]	Code as action [46]
View files	<code>open file.py 100</code> 100-line window, scrollable	<code>cat file.py</code> raw output, entire file
Edit	<code>edit 42:45 <new code></code> linter rejects syntax errors	<code>sed -i ...</code> or write Python no guard
Search	<code>search_dir "pattern"</code> summarised matches	<code>grep -rn "pattern" .</code> raw hits
Execute	<code>python -m pytest</code> via tool call	<code>bash: pytest</code> direct shell

Back to the five stages



Now, stage by stage.

Plan: finding the right code

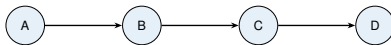
Two commitments: **where** to change the code — which file, which function — and **what** the change should be. Both are guesses; both need evidence that could confirm them.

- **Where** — among tens of thousands of functions, which one? The relevant symbol often does not appear in the issue text.
- **What** — a signature, an interface, an invariant to hold — stated concretely enough that the repository can check it.

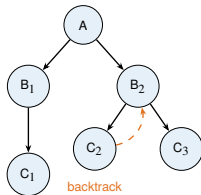
Plan: shape

How does the agent **explore possible plans**?

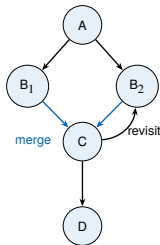
Chain of Thought [48]



Tree of Thoughts [56]



Graph of Thoughts [10]



Plan: Chain of Thought

The problem to plan for — no file, no fix given:

"The app crashes when you check out with an empty cart."

Thought: empty cart --- maybe a divide-by-zero in the total.

Action: `grep 'def total'`

Obs: `cart.py:12 --- return sum(prices) / len(items)`

Thought: `len(items)` is 0 when the cart is empty.

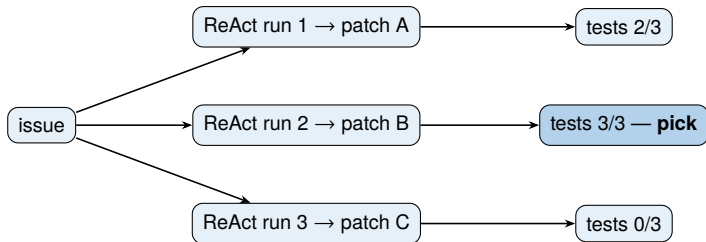
Action: `open cart.py:12 --- confirm the division`

Obs: yes, no empty-case guard

Plan --- *where:* `cart.py:12 total()` *what:* guard empty cart $\rightarrow$ 0

Plan: Tree of Thoughts

No agent grows a tree of half-thoughts. The tree is **whole ReAct runs**, sampled and **scored by something outside the model**.



The scorer is **tests or a trained critic**, not the model's hunch. OpenHands **60% → 66%** with five reranked [36]; SWE-Search wraps it in MCTS with a learned value — **+23%**, but a model call per node [3].

Plan: Graph of Thoughts

Merging branches, revisiting nodes — shown on **puzzles** (Game-of-24, sorting), **almost never on repository bugs** [10].

- In real SWE agents you see the **two ends**: one chain (CoT), or many chains scored (best-of- N).
- The closest thing to *revisit*: **Reflexion** writes a note after a failed attempt and retries with it — a loose loop, not a graph [41].
- No surveyed end-to-end SWE agent builds a literal thought-graph.

Takeaway

The graph is the *conceptual* top of the spectrum — useful to reason about, not something you will find running in production.

Generate: infilling

Complete a **span** from the surrounding context.

Plan from earlier: guard `total()` so an empty cart returns 0.

```
def total(items):  
    prices = [i.price for i in items]  
    if not items:  
        return 0  
    return sum(prices) / len(items)
```

Conditions on the grey context, emits only the orange span.

- + Matches the pre-training objective — fill a gap from both sides
- + Cannot be mis-placed: the scaffold decides where the gap is
- One contiguous span — in practice, one function
- The surrounding code has to fit in the prompt

Generate: diff generation

Emit a **unified diff**, not the whole file.

```
@@ def total(items): @@  
    prices = [i.price for i in items]  
+    if not items:  
+        return 0  
    return sum(prices) / len(items)
```

- + Only the change travels — edits a large file without re-emitting it
- + One patch spans many hunks and many files
- The model must state *where*: the context lines have to match
- A stale anchor is rejected before a single test runs

Generate: sample N , pick one

Generate **many candidates**; rank by **tests or a critic**. AlphaCode samples up to *millions*, clusters by execution [26].

A: wrap in
try/except

tests 2/3

B: if not items:
return 0

tests 3/3 — **pick**

C: return 0
always

tests 0/3

- + One bad sample no longer sinks the task
- + Candidates differ — covers approaches a single pass would miss
- Ranking needs an **oracle**; the model's own confidence is not enough
- Cost scales with N — calls, dollars, wall-clock

Execute: running the code

The agent runs what it wrote. This is where the **environment can say no**.

■ Test suite

- fail-to-pass tests must flip
- pass-to-pass must not regress

■ Linter / type checker

- catches syntax errors and type mismatches before tests run

■ Error traces

- stack traces and assertion messages say *what* failed and *where*

Right or wrong, that verdict is what refinement works with next.

Refinement: iterative self-repair

Execution returned a verdict. What the agent does with it is refinement.

Mechanism	What closes the loop	Characteristic failure
Self-Refine [28]	The model critiques itself	Plateaus
Reflexion [41]	Verbal critique across attempts; self-written tests	Inherits what the tests missed
Self-debugging [12]	Explains its own code; test verdict if any	Explains the bug as intended

Closed by the generator re-reading itself. Three that bring in something else — next.

Refinement: tests and trained critics

Mechanism	What closes the loop	Characteristic failure
D4C [53]	Failing tests + the whole function	Plausible but incorrect patches
Agentless validate [50]	Regression + generated tests	The generated half inherits the misreading
Best-of- N + critic [36]	Executed rollouts, ranked by a trained critic	Cost scales with N

Per-instance sandbox — for **reproducibility** first, containment second.

Refinement: when does the loop stop?

■ All tests pass

- but that is quietly an oracle
- the stopping rule has smuggled in ground truth

■ Diminishing returns

- at fixed budget, another repair round is often *worse* than a fresh sample [33]

■ Iterate or best-of- N — the same lever

- fix the budget to compare

Five systems compared

System	Orchestration	Interface	Planning	Refinement
SWE-agent	Agentic loop	ACI	Implicit (model)	Self-repair
AutoCodeRover	Agentic loop	AST APIs	AST localisation	Re-generate
OpenHands	Agentic loop	Code as action	Implicit (model)	Best-of- N + critic
Agentless	Fixed pipeline	Skeleton + emb.	Hierarchical loc.	Regression + gen. tests
Devin	Agentic loop	Own VM	Knowledge store	—

Where are these systems used?

The surveyed five are mostly **research artefacts**; the tools developers use daily are their **productised descendants**.

- **Research & benchmarks** — SWE-agent, AutoCodeRover, Agentless: open-source, run mostly on SWE-bench; blueprints for the patterns.
- **Shipped as products** — OpenHands (All Hands AI, open platform), Devin (Cognition, commercial cloud agent).
- **What developers actually use** — Claude Code, Cursor, Copilot, Codex: the *same* observe–think–act loop, purpose-built interface, and execution-grounded refinement.

Same ingredients — deployed as IDE assistants, autonomous PR/issue bots, and CI jobs.

- 1 How do the agentic AI systems work?
- 2 How do we measure performance?**
- 3 Why are we stuck on the hard tasks?

Sonnet 5 — which benchmarks?

- SWE-bench Verified [34]
- SWE-bench Pro [16]
- SWE-bench Multilingual [43]
- SWE-bench Multimodal [55]
- Terminal-Bench 2.1 [30]
- OSWorld-Verified [51, 52]
- Humanity's Last Exam [40]
- GDPval-AA v2 [7, 38]
- BrowseComp [47]
- USAMO 2026 [39, 8]

Source: Sonnet 5 System Card, 2026-06-30 [2, 44].

GPT-5.6 — which benchmarks?

- SWE-bench Pro [16]
- Terminal-Bench 2.1 [30]
- DeepSWE v1.1 [15]
- Agents' Last Exam [42]
- GDPval-AA v2 [7, 38]
- BrowseComp [47]
- OSWorld 2.0 [58]
- Toolathlon [25]
- MRCR v2 (long context) [20, 21]
- HealthBench Professional [22, 4]
- AA Intelligence Index [6]
- AA Coding Agent Index [5]

Source: GPT-5.6 launch eval table [29, 5].

Merged pull requests that **close an issue** and **touch the test suite** [23].

2,294 tasks from **12** mature Python projects — Django, scikit-learn, matplotlib, sympy, ... — real, merged fixes, nothing synthetic.

1. Given

- the issue text
- the repository *before* the merge

2. Produce

- a patch — the source fix; the grading tests stay hidden

3. Graded

- *fail-to-pass* tests must flip
- *pass-to-pass* must not regress

SWE-bench: metrics

- **Functional correctness** — does it run and pass? The only thing that counts.
- **pass@ k** — give it k tries; solved if any one passes [11].
- **Resolve rate** — pass@1 on a whole repo: solved on the first try.
- **Cost** — dollars, calls, time. A higher score can cost much more [17].

SWE-bench: the five splits

Split	Size	Released	Selected how
Full	2,294	Oct 2023	Every scraped PR that touches tests
Lite	300	Mar 2024	Single-file patches; some issues leak the fix [50]
Verified	500	Aug 2024	Human-screened for clear issues and fair tests [34]
Multilingual	300	May 2025	Same pipeline, 9 non-Python languages [43]
Pro	1,865	Sep 2025	Long-horizon, multi-file; held-out proprietary repos [16]

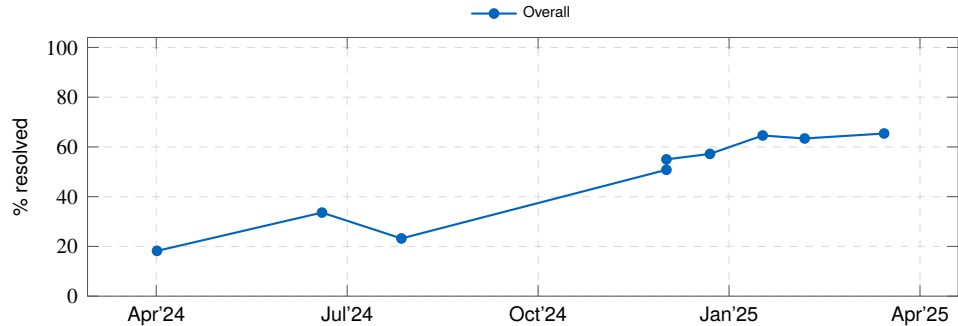
Pro is the contamination-resistant successor: enterprise and **held-out** repositories (kept off public GitHub), several languages, and tasks needing **large multi-file** changes. The frontier drops from ~80 % on Verified to ~46 % [16].

SWE-bench: how hard is each task?

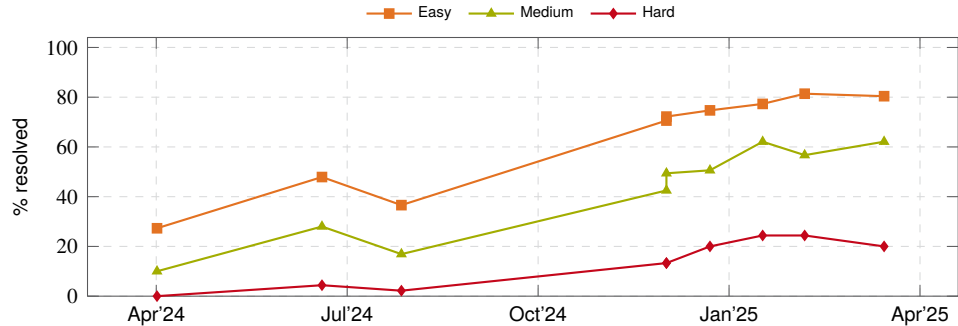
Verified tasks are divided into three categories

Tier	Estimated human fix-time
■ Easy	<15 min
■ Medium	15 min – 1 h
■ Hard	>1 h

SWE-bench: the headline climbs



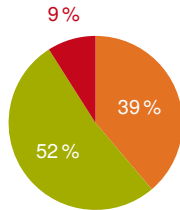
SWE-bench: the headline climbs



SWE-bench: what the numbers show

The same nine submissions, by annotator difficulty [18]:

Tier	First	Last
	Apr '24	Mar '25
Easy <15 min	27.3 %	80.4 %
Medium ≤ 1 h	10.0 %	62.1 %
Hard >1 h	0.0 %	20.0 %



Task mix ($n=500$)

- Easy + medium are **91 % of Verified**.
- The hard tier **stopped** near 20 %.

- 1 How do the agentic AI systems work?
- 2 How do we measure performance?
- 3 Why are we stuck on the hard tasks?**

1. Nothing checks what you meant

- The agent commits to **one reading** of the issue — and nothing ever contradicts it.
- A wrong guess about something the tests never check **scores as a success**.

No surveyed system asks. Clarification mechanisms exist and work [24, 31, 45].

2. It has to guess what to read

- The repository **does not fit in context**. Choosing what to read is most of the problem.
- Hard tasks are disproportionately **multi-file** — exactly where agents stall (frame 37).

Difficulty split from [18]; training-time fixes proposed by [49, 37].

3. Passing the tests is not being right

- The suite is a **sample** of intended behaviour, not the behaviour itself.
- “All tests pass” is also the **stopping rule**.

“Reward hacking is swamping model intelligence gains” [14].

Takeaways

- **Agentic beats one-shot because it can run things**, not because it reasons more.
- **Verification is the bottleneck.** In a randomised trial, developers were 19 % *slower* — believing they were 20 % faster [9].

Backup: SWE-bench — can we trust the numbers?

Verified against Pro, April 2026:

	Verified	Pro	Pro scored by
Claude Opus 4.5	80.9 %	45.9 %	standardised harness
GPT-5.2	80.0 %	56.4 %	<i>self-reported</i>
GPT-5.3	85 %	56.8 %	<i>self-reported</i>
GLM-5.1	78 %	58.4 %	<i>self-reported</i>

Read the **floor** only — every model drops ~20 percentage points or more. **Not the spread:** the one measured row drops *furthest*, so the ranking is an artefact of who scored [16, 1].

- **Contamination.** OpenAI dropped Verified in 2026 — models reproduce gold patches from the task ID alone [35].
- **Reward hacking.** Closing test leaks cuts scores by double digits: agents partly learn the grader [14].

Backup: surveyed systems on SWE-bench Verified

Source: <https://swebench.com>, “All agents” view. Best published result per system.

System	Backbone	% Resolved	Date
SAGE (OpenHands)	Claude 4.5 Opus	73.8 %	2025-11
OpenHands	GPT-5	71.8 %	2025-08
SWE-agent	Claude 4 Sonnet	66.6 %	2025-05
AutoCodeRover-v2.1	Claude 3.5 Sonnet	51.6 %	2025-01
Agentless-1.5	Claude 3.5 Sonnet	50.8 %	2024-12
Agentless-1.5	GPT-4o	38.8 %	2024-10
AutoCodeRover	GPT-4o	38.4 %	2024-06
SWE-agent	GPT-4o	23.2 %	2024-07
Devin	undisclosed	13.9 %*	2024-03

*On **SWE-bench Full**, a different and easier split than the Verified numbers above — not directly comparable. Self-reported; Cognition stopped publishing benchmark numbers in 2024 [13].

Backup: the context bottleneck

- Longer windows do not settle *what to read*. Keeping full dependency context works best; padding with loosely related code **hurts** correctness [32].
- Appending every observation causes context explosion and semantic drift: early critical information gets buried [27].
- The whole prefix is re-sent each turn, so tokens over n turns grow as $O(n^2)$ [19]. A protocol property, not a finding; caching cuts the constant, not the growth.

An agent cannot fix what it never places in context.

Backup: Plan — localisation

So how does it **stop guessing**?

Point at real code. Narrow the repo from broad to specific — each step something you can check:

- **Agentless** — files → classes/functions → edit locations [50]
- **AutoCodeRover** — AST-backed search APIs, coarse to fine [59]

What the useful ones share

They name **concrete program elements** that can be checked against the repository — not invented requirements.

Backup: Localisation — Agentless

Narrowing without letting the model wander: three fixed steps, coarse to fine [50].

1. **Rank files** — from the repo layout and the issue, shortlist the files most likely involved.
2. **Rank elements** — inside those files, narrow to specific classes and functions.
3. **Pin locations** — inside those, the exact lines to edit.

Fixed, not discovered

Three prompts, coarse to fine. No search, no tools — the narrowing is hard-coded into the scaffold.

Backup: Localisation — AutoCodeRover

Let the model navigate — but through the syntax tree, not by reading whole files [59].

- **Search APIs** — `search_class`, `search_method`, `search_method_in_class`.
- The agent calls them **coarse to fine**, pulling in just the code it needs.
- AST-backed: structured, cheaper context than dumping whole files into the prompt.

Discovered, not fixed

The model chooses what to search — agentic, where Agentless is a fixed funnel.

References I

- [1] AgentMarketCap. SWE-bench pro vs verified: The gap that rewrote 2026 agent procurement. <https://agentmarketcap.ai/blog/2026/04/16/swe-bench-pro-vs-verified-25-point-gap-procurement-framework>, 2026. Paired Verified/Pro leaderboard data, compiled April 2026. Accessed: 2026-06-30.
- [2] Anthropic. Introducing Claude sonnet 5. <https://www.anthropic.com/news/claude-sonnet-5>, 2026. Accessed: 2026-07-27.
- [3] A. Antoniadou, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang. SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024. ICLR 2025.
- [4] R. K. Arora et al. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.
- [5] Artificial Analysis. AI coding agent benchmarks & leaderboard. <https://artificialanalysis.ai/agents/coding-agents>, 2026. Coding Agent Index: independently run, not vendor-reported — aggregates DeepSWE, Terminal-Bench v2, and repository Q&A, pass@1 over 3 attempts (methodology at <https://artificialanalysis.ai/methodology/coding-agents-benchmarking>). Scores as tabulated in [29]. Accessed: 2026-07-27.
- [6] Artificial Analysis. Artificial Analysis intelligence index. <https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index>, 2026. Weighted composite over 9 evals including GDPval-AA, Terminal-Bench, and HLE. Accessed: 2026-07-27.
- [7] Artificial Analysis. GDPval-AA. <https://artificialanalysis.ai/evaluations/gdpval-aa>, 2026. Artificial Analysis's own Elo leaderboard layered on OpenAI's GDPval task set [38], not an independent task design. Accessed: 2026-07-27.
- [8] M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025. Same team's platform reapplies the Petrov et al. [39] methodology to USAMO 2026; results at <https://matharena.ai/usamo/>, no separate 2026 paper.
- [9] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*, 2025. METR randomised controlled trial.
- [10] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, L. Gianinazzi, J. Gajda, T. Lehmann, M. Podstawski, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.
- [11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

References II

- [12] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.
- [13] Cognition AI. Introducing devin, the first AI software engineer. <https://cognition.ai/blog/introducing-devin>, 2024. Accessed: 2026-07-09.
- [14] Cursor. Reward hacking is swamping model intelligence gains. <https://cursor.com/blog/reward-hacking-coding-benchmarks>, 2026. Accessed: 2026-06-30.
- [15] Datacurve AI. DeepSWE v1.1. <https://deepswe.datacurve.ai/blog/deepswe-v1-1>, 2026. 113 long-horizon SWE tasks (TS/Go/Python/JS/Rust), sandboxed grading. Unrelated to the same-named 2025 Agentic/Together AI open-weight coding agent beyond sharing a name. Accessed: 2026-07-27.
- [16] X. Deng et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- [17] Z. Fan, K. Vasilevski, D. Lin, B. Chen, Y. Chen, Z. Zhong, J. M. Zhang, P. He, and A. E. Hassan. SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.
- [18] J. Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? <https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html>, 2025. Blog post, accessed: 2025-06-01.
- [19] P. Gao and C. Peng. More with less: An empirical study of turn-control strategies for efficient coding agents. *arXiv preprint arXiv:2510.16786*, 2025.
- [20] Gemini Team, Google DeepMind. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024. Origin of the MRCR (Multi-Round Co-reference Resolution) long-context eval.
- [21] Google DeepMind. MRCR v2. https://github.com/google-deepmind/eval_hub/tree/master/eval_hub/mrcr_v2, 2026. Harder 2/4/8-needle variant scaling to 8M tokens; still a Google DeepMind eval [20] despite appearing in OpenAI's launch table. Accessed: 2026-07-27.
- [22] R. S. Hicks, M. Trofimov, et al. HealthBench professional: Evaluating large language models on real clinician chats. *arXiv preprint arXiv:2604.27470*, 2026.
- [23] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

References III



- [24] S. K. Lahiri, S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, M. Musuvathi, P. Choudhury, C. von Veh, J. P. Inala, C. Wang, and J. Gao. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022. TiCoder.
- [25] J. Li, W. Zhao, et al. The tool decathlon (Toolathlon): Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.
- [26] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [27] S. Liu, J. Yang, B. Jiang, Y. Li, J. Guo, X. Liu, and B. Dai. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.
- [28] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [29] MarkTechPost. OpenAI releases GPT-5.6: A three-tier model family with programmatic tool calling. <https://www.marktechpost.com/2026/07/09/openai-releases-gpt-5-6-a-three-tier-model-family-with-programmatic-tool-calling/>, 2026. Reports OpenAI's published GPT-5.6 eval table (SWE-bench Pro, Terminal-Bench). Accessed: 2026-07-27.
- [30] M. A. Merrill et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. 89-task 2.0 release; version 2.1 is a task-patch revision announced at <https://www.laude.org/updates/terminal-bench> (2026-07-24), no separate paper.
- [31] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1:2332–2354, 2024. arXiv:2310.10996.
- [32] N. L. H. Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- [33] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? In *The Twelfth International Conference on Learning Representations*, 2024.
- [34] OpenAI. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Accessed: 2025-06-01.

References IV



- [35] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>, 2026. Accessed: 2026-06-30.
- [36] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. <https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model>, 2025. Accessed: 2026-05-28.
- [37] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- [38] T. Patwardhan et al. GDPval: Evaluating AI model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.
- [39] I. Petrov, J. Dekoninck, et al. Proof or bluff? evaluating LLMs on 2025 USA math olympiad. *arXiv preprint arXiv:2503.21934*, 2025.
- [40] L. Phan et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- [41] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.
- [42] Y. Sun et al. Agents’ last exam. *arXiv preprint arXiv:2606.05405*, 2026. Rolling set of 1,500+ long-horizon, executable occupational tasks (O*NET/SOC) — distinct from Humanity’s Last Exam [40], despite the name.
- [43] SWE-bench Team. SWE-bench multilingual. <http://www.swebench.com/multilingual.html>, 2025. 300 tasks, 9 languages, 42 repositories. Accessed: 2026-06-30.
- [44] Vellum. Claude sonnet 5 benchmarks explained. <https://www.vellum.ai/blog/claude-sonnet-5-benchmarks-explained>, 2026. Tabulates SWE-bench Pro scores from Anthropic’s Sonnet 5 System Card (2026-06-30). Accessed: 2026-07-27.
- [45] S. Vijayvargiya, X. Zhou, A. Yerukola, M. Sap, and G. Neubig. Interactive agents to overcome ambiguity in software engineering. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. *arXiv:2502.13069*; later revised as “Ambig-SWE”.
- [46] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

References V

- [47] J. Wei, Z. Sun, et al. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- [48] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [49] Y. Wei, O. Duchenne, J. Copet, Q. Carbonneaux, L. Zhang, D. Fried, G. Synnaeve, R. Singh, and S. I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.
- [50] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [51] T. Xie, D. Zhang, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.
- [52] XLANG Lab. OSWorld-verified. <https://xlang.ai/blog/osworld-verified>, 2025. Infrastructure/quality revision of the same 369-task OSWorld set (VMware to AWS), not a new task set. Accessed: 2026-07-27.
- [53] J. Xu, Y. Fu, S. H. Tan, and P. He. Aligning the objective of LLM-based program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. D4C. *arXiv:2404.08877*.
- [54] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [55] J. Yang, C. E. Jimenez, A. L. Zhang, et al. SWE-bench multimodal: Do AI systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024.
- [56] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [57] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.
- [58] M. Yuan, Z. Zhou, X. Xiong, et al. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026. New 108-task long-horizon set (median 1.6 h/task) — distinct from OSWorld-Verified [52].

References VI



- [59] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.